# Does the Shortest Path Matter?

# Ford–Fulkerson (DFS) versus Edmonds–Karp (BFS) for Maximum Flow

**Theme 3:** Theoretical-Optimal vs Practical

## Summary of what I found


- **Same method, different path choice:** Ford–Fulkerson (FF) takes the first augmenting path a depth-first search finds; Edmonds–Karp (EK) takes a fewest-edges path found by breadth-first search
- **EK wins on random graphs:** It was 13.5 times faster on a sparse graph (1,600 vertices) and 111.7 times faster on a dense one (100 vertices). FF pushed 12.5 to 20 times more paths, and its paths were 24 to 177 times longer.
- **Capacity values matter in practice:** As the largest capacity C grew from 1 to 10,000, EK pushed 3 to 12 paths, while FF went from 3 to 4,442 paths and became 452 times slower
- **The simpler algorithm wins on unit-capacity matching (up to 2.6 times faster):** On a matching network with capacities 1 to C, I found the crossover near C = 10
- **Main surprise:** Time crosses over much earlier than arcs scanned predict. The longer paths FF pushes explain most of the gap (cost-model error 41% down to 24%), but not all of it


## 1. The problem and the algorithms

### 1.1 The problem

A flow network is a directed graph with a source s, a sink t, and a non-negative capacity on every edge. A flow gives every edge an amount that never exceeds its capacity, and at every vertex other than s and t the amount flowing in equals the amount flowing out. The value of a flow is the amount leaving s. The maximum-flow problem asks for a flow of largest value.

In this project, capacities are whole numbers, because with irrational capacities Ford–Fulkerson may never stop. My code rejects non-integer capacities, and a unit test checks this

### 1.2 The method both algorithms share

Both algorithms use the residual network. For an edge from u to v with capacity c carrying flow f, the residual network has a forward edge with spare capacity $c - f$ and a backward edge with capacity f. The backward edge lets a later path undo flow that an earlier path committed. An

augmenting path is any s–t path in the residual network. Its bottleneck is the smallest residual capacity on the path

The Ford–Fulkerson method is: start with zero flow; while an augmenting path exists, push its bottleneck amount along it; stop when none exists. When it stops, the flow is maximum, because the vertices reachable from s then form a cut with exactly the same capacity as the flow (max-flow = min-cut)

It is a method, not an algorithm, because it does not say which augmenting path to take, and different choices give different running times. That is exactly the freedom my two algorithms use differently

A small example shows why backward edges matter. On a six-vertex test network, always pushing along the widest path without ever undoing flow gets stuck at value 18, while the true maximum is 21. A unit test confirms that the greedy rule gets 18 and that both of my algorithms reach 21

**1.3 Algorithm A: Ford–Fulkerson with depth-first search (Nguyen Phuong Anh Le)**

- **Key idea:** Take the first augmenting path a depth-first search happens to find. The search follows one arc as deep as it can, backs up from dead ends, and stops when it reaches t. Arcs are tried in the order they are stored
- **Implementation:** The search is iterative (it uses a stack, not recursion), so very long paths cannot hit Python's recursion limit. A test pushes flow along a 5,000-edge path to prove this. After the search, the code walks back from t to s, finds the bottleneck, subtracts it on forward arcs and adds it on the paired backward arcs
- **Complexity:** With integer capacities every push raises the flow value by at least 1, so there are at most F augmentations, where F is the value of the maximum flow. Each augmentation costs O(E), so the total is O(FE). This bound depends on the capacity values, because F can be huge: a capacity of $10^9$ fits in 30 bits but can force about $10^9$ augmentations if paths are chosen badly
- **What is different about it**: There is no guarantee about the path. A depth-first path can be long and winding

**1.4 Algorithm B: Edmonds–Karp with breadth-first search (Nguyen Phuong Anh Le)**

- **Key idea:** Always take an augmenting path with the fewest edges. A breadth-first search from s explores the residual network level by level, so the first time it reaches t it has found such a path
- **Why it is polynomial:** Let the level of a vertex be its BFS distance from s. A shortest-path augmentation can only create new residual edges that point one level backward, so the distance from s to t never decreases. Each augmentation saturates at least one edge that goes one level forward. There are at most E such edges for each distance value and at most V distance values. So EK performs O(VE) augmentations, and

with an O(E) search each, $O(VE^2)$ in total. This bound does not depend on the capacities at all

## 2. Theoretical comparison and predictions

### 2.1 What the bounds say, and when each one is better

The two bounds cross over when F is about VE. If the maximum flow value is small, FF's bound O(FE) is the smaller one. For unit-capacity bipartite matching, F is at most the number of vertices, so FF's bound is O(VE) while EK's is $O(VE^2)$. On paper FF should be competitive there. If capacities are large, F can be enormous, and EK's capacity-independent bound is far smaller. So the theory predicts a regime in which the simpler algorithm is good enough and a regime in which the guarantee matters. That is the gap between theory and practice this project tries to locate

A worst-case bound is only an upper limit. Whether real inputs come anywhere near it is an empirical question

### 2.2 Constant factors and memory

Both algorithms use O(V + E) memory: the paired-arc residual network plus a few arrays of length V per search. Per search, FF allocates two arrays of length V (the arc used to reach each vertex and the next arc to try), while EK allocates one array plus a queue. A search costs time proportional to the number of arcs it examines. Pushing a path costs time proportional to the path's length. EK's paths are short, because BFS finds the shortest one. DFS paths can be very long, so the cost of pushing them may matter

### 2.3 Input features each algorithm should be sensitive to

- **FF:** capacity values (through F), the number of vertices (long depth-first paths), and the order in which arcs are stored
- **EK:** the number of vertices and edges, but not the capacity values

### 2.4 My cost idea before running

Running time should grow with (number of augmentations) × (arcs scanned per search). A DFS may scan fewer arcs per search because it stops at the first path, but it may need more and longer augmentations. I did not include a separate cost for the length of pushed paths; I only added that after seeing the results (Section 5)

### 2.5 Predictions

| ID | Prediction made before running | Where it is checked |
|---|---|---|
| P1 | Random sparse and dense graphs (largest size): EK is faster, FF does more augmentations, and FF's average path is at least 2 times longer | random_sparse, random_dense |
| P2 | Density sweep (n = 60, p from 0.03 to 0.9): for p ≥ | density_sweep |

| | | |
|---|---|---|
| | 0.12, FF's average path stays at least 0.3n while EK's stays at most 5, and the FF/EK time ratio is larger at the densest p than at the sparsest | |
| P3 | Unit-capacity matching (largest size): both algorithms push almost the same number of paths (within 10%), and FF is faster | matching_unit |
| P4 | Capacity growth on random sparse graphs: FF's augmentations grow at least 10 times from C = 1 to C = 10,000, while EK's grow at most 3 times | capacity_sweep |
| P5 | Crossover on the matching network (k = 200): FF faster at C = 1, EK faster at C = 1000, with the crossover between C = 5 and C = 100 | matching_crossover |
| P6 | On at least 80% of instances, the algorithm that scans more arcs in total is also the slower one | all instances |

## 3. Experimental methodology

### 3.1 Input families and why I chose them

Every input is generated from a fixed random seed (seeds 0 to 4), so the same inputs appear in every run. Generation is never timed

| Family | What it is | What varies | Why it is in the study |
|---|---|---|---|
| random_sparse | n vertices, about 4n random edges plus one random s–t path; capacities 1..100 | n = 100 to 1,600 (up to 6,400 edges) | basic sparse comparison |
| random_dense | each ordered pair is an edge with probability 0.3; capacities 1..100 | n = 20 to 100 (up to 3,035 edges) | basic dense comparison |
| density_sweep | same generator, n=60 | p = 0.03 to 0.9 | does a DFS find the sink faster in denser graphs? |
| matching_unit | bipartite matching network, 4 random neighbours per left vertex, all capacities 1 | k = 50 to 800 (up to 1,602 vertices, 4,800 edges) | every path carries one unit, so F is small |
| capacity_sweep | random sparse graph, n = 300, same shape each | C = 1 to 10,000 | do capacity values matter in practice? |

| | | | |
|---|---|---|---|
| | time | | |
| matching_crossover | matching network, k = 200, capacities 1..C | C = 1 to 1,000 | locates the crossover |

This gives 170 instances. I chose families so that each one isolates one input feature: size, density, unit capacities, or capacity values. EK's theoretical worst case needs a specially built graph that I did not construct. FF's capacity-dependent worst case is shown by a hand-traced test (the four-vertex "diamond" network, where badly chosen paths force 2M augmentations) and, in practice, by the capacity sweep

**3.2 What I measure**

For every run I record the running time and three machine-independent counters: the number of augmentations (paths pushed), the number of arcs scanned while searching, and the total length of the pushed paths. The counters let me explain the timings instead of only reporting them

**3.3 Checking answers before timing**

Before any timing, both algorithms solve every instance and each answer is checked without trusting how it was found: the capacity and conservation rules hold, and a cut with exactly the same capacity as the flow exists (max-flow = min-cut), read off the residual network as the vertices reachable from s. The two flow values must be equal, or the benchmark stops. Separately, 19 unit tests compare both algorithms with a brute-force minimum cut on 300 random tiny graphs, reproduce textbook examples (greedy gets 18 but the maximum is 21, the minimum cut {s, 1}, Kőnig's theorem on small bipartite graphs, a matrix-rounding puzzle), and cover edge cases such as source equal to sink

**3.4 Timing procedure and hardware**

For each instance and each algorithm: one untimed warm-up run, then 5 timed runs. The algorithm that goes first alternates from run to run, and Python's garbage collector is switched off while the clock runs. I use time.perf_counter. Every single timing is saved. The final run used Python 3.14.3 on macOS 15.4.1, Intel x86_64, 8 logical CPUs, on 30 October 2026. This gives 1,700 timed runs. The computer is a MacBook Air (Retina, 13-inch, 2020) with a 1.1 GHz quad-core Intel Core i5 processor (8 logical CPUs) and 8 GB of memory (LPDDR4X, 3733 MHz)

In the plots, the line is the median over seeds of the per-instance median time. The bars run from the smallest to the largest time over all seeds and repetitions, so a wide bar partly reflects that some random graphs are harder than others, not only timing noise. I use medians because single slow runs do occur (Section 4.8)

**3.5 Fairness**

Both algorithms receive identical instances and use the same network data structure (paired arc arrays). Each is about 60 lines of plain Python, the augmenting step is written the same way, and neither is tuned. Both are measured on the algorithm only: input generation and file output are outside the timed region

### 3.6 Reproducibility

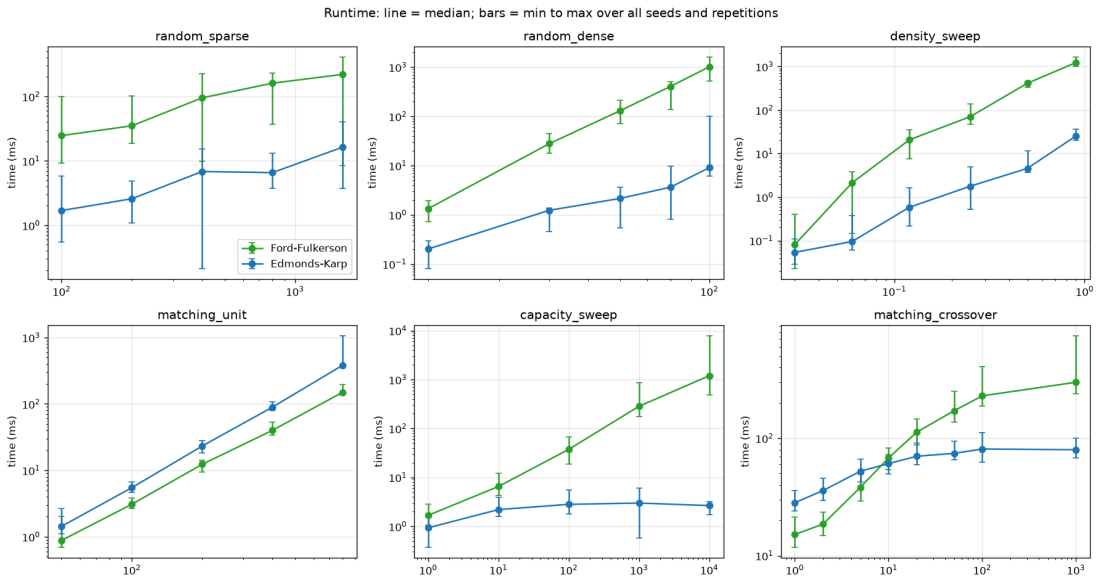
One command, python3 run_all.py, runs the tests, the benchmark, the analysis and the cost-model check. Raw timings (raw_trials.csv), every instance with its counters (instances.csv) and the machine details (metadata.json) are saved. Every table and figure in this report is generated from those files by committed scripts

### 3.7 Limits of the study

- Pure-Python timings include interpreter overhead, so I rely on the counters for explanations
- Sizes in the main experiments are small because FF is slow (about one second for a dense graph with 100 vertices). Larger sizes were not tested, so I cannot say whether the trends continue beyond them. Everything ran on one machine
- Run-to-run noise is about 20%. The same inputs appear in matching_unit (k = 200) and matching_crossover (C = 1), and their times differ (FF 12.3 against 15.1 ms, EK 23.2 against 28.2 ms) while their ratios are nearly equal (0.53 and 0.54). Ratios are more reliable than single timings, and a ratio close to 1 should not be over-interpreted
- The crossover was measured for one size (k = 200) and one degree (4)

## 4. Results and analysis

Times are medians in milliseconds. Ratios are FF divided by EK, so a ratio above 1 means FF is slower. “Paths” is the number of augmentations, “avg path” is the average number of edges on a pushed path, and “scans” is the total number of arcs examined during searches. All numbers come from results/summary.md




### 4.1 Random graphs: EK wins by a wide margin

| Family | Size | FF ms | EK ms | FF/EK | FF paths | EK paths | FF avg path | EK avg path |
|---|---|---|---|---|---|---|---|---|
| random_sparse | n=100 | 24.7 | 1.7 | 14.6 | 153 | 12 | 73.1 | 5.4 |
| random_sparse | n=400 | 95.2 | 6.8 | 13.9 | 140 | 13 | 298.4 | 6.0 |
| random_sparse | n=1600 | 220.3 | 16.3 | 13.5 | 150 | 12 | 1275.8 | 7.2 |
| random_dense | n=20 | 1.3 | 0.2 | 6.6 | 45 | 10 | 13.2 | 3.5 |
| random_dense | n=60 | 129.8 | 2.1 | 60.4 | 466 | 28 | 46.6 | 2.9 |
| random_dense | n=100 | 1014.0 | 9.1 | 111.7 | 1280 | 64 | 68.7 | 2.9 |

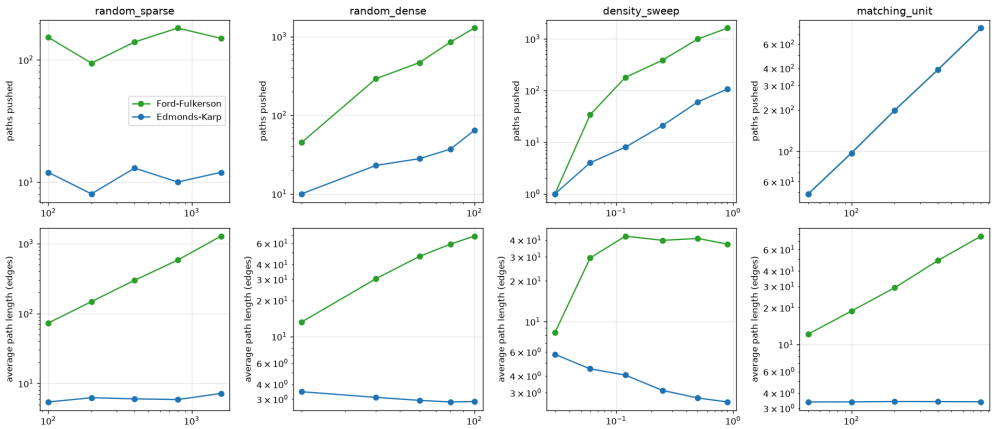
- **What it shows:** EK was faster at every size I tried. On sparse graphs, the gap stayed at roughly 13 to 24 times. On dense graphs, it grew steadily with size, from 6.6 times at n = 20 to 111.7 times at n = 100
- **Why:** The counters tell the story. FF pushes 12.5 times more paths on the largest sparse graph and 20 times more on the largest dense one, and its paths are far longer: on the sparse graph with 1,600 vertices its average path has 1,276 edges, against 7.2 for EK. A depth-first search returns the first path it finds, and in a random graph where most vertices are reachable, it wanders through a large part of the graph before reaching t. A breadth-first search stops at the shortest path, which is only a handful of edges. FF's path length grows almost in proportion to n (73 edges at n = 100, 1,276 at n = 1,600), while EK's stays between 5 and 7

- One detail goes against my first expectation that a DFS is cheap per search. Dividing total scans by the number of searches (paths + 1) from the table, FF scans about 3,544 arcs per search on the largest sparse graph, against 7,753 for EK, so a DFS search is about 2.2 times cheaper there. On the largest dense graph, it is the other way round: about 3,190 arcs per search for FF and 1,389 for EK. So the DFS is only cheaper per search on sparse graphs, and its advantage per search is far too small to pay for 12 to 20 times more paths

### 4.2 Density: FF's paths stay long, EK's get shorter

| p (n=60) | FF ms | EK ms | FF/EK | FF paths | EK paths | FF avg path | EK avg path |
|---|---|---|---|---|---|---|---|
| 0.03 | 0.08 | 0.05 | 1.5 | 1 | 1 | 8.3 | 5.8 |
| 0.06 | 2.13 | 0.10 | 22.4 | 34 | 4 | 29.5 | 4.5 |
| 0.12 | 20.7 | 0.58 | 35.6 | 179 | 8 | 42.6 | 4.0 |
| 0.25 | 70.2 | 1.77 | 39.6 | 387 | 21 | 39.7 | 3.1 |
| 0.5 | 412.3 | 4.57 | 90.3 | 990 | 60 | 41.0 | 2.7 |
| 0.9 | 1232.2 | 25.3 | 48.7 | 1613 | 107 | 37.2 | 2.6 |

- **What it shows:** With the number of vertices fixed at 60, making the graph denser makes FF relatively much worse: the ratio rises from 1.5 to 90.3 between p = 0.03 and p = 0.5. As the graph gets denser, FF's average path stays at about 37 to 43 edges, roughly two-thirds of all vertices, while EK's average path shrinks from 5.8 to 2.6 edges, because more shortcuts exist
- **Why:** A denser graph offers BFS more shortcuts, so its paths get shorter. A DFS ignores shortcuts and keeps going deeper, so its paths stay long. More edges also mean more flow to push, so both algorithms push more paths, but FF pushes about 15 to 17 times more at the dense end (1,613 against 107 at p = 0.9)



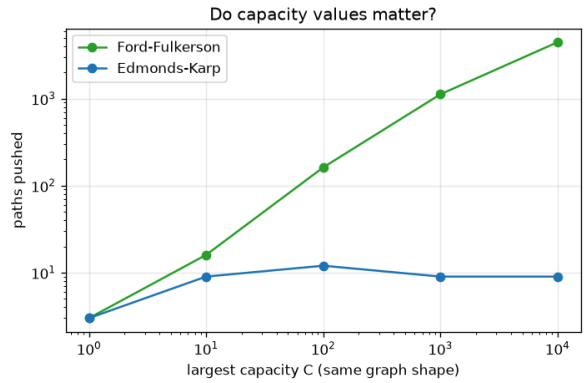
### 4.3 Unit-capacity matching: the simpler algorithm wins

| k | FF ms | EK ms | FF/EK | Paths (both) | FF scans | EK scans |
|---|---|---|---|---|---|---|
| 50 | 0.88 | 1.44 | 0.61 | 49 | 3,510 | 12,077 |
| 200 | 12.3 | 23.2 | 0.53 | 198 | 43,117 | 178,967 |
| 800 | 149.7 | 382.4 | 0.39 | 783 | 540,824 | 2,877,797 |

- **What it shows:** Here FF is faster at every size, and its advantage grows with size, from 1.6 times faster at k = 50 to 2.6 times faster at k = 800
- **Why:** With unit capacities, every path carries exactly one unit, so both algorithms push the same number of paths, equal to the matching size (783 at k = 800). The path count cannot hurt FF here, and what remains is the cost of one search. Dividing total scans by searches (paths + 1), FF scans about 690 arcs per search against about 3,671 for EK, because a BFS has to explore a whole frontier before it reaches t. This is the regime where the algorithm with the weaker worst-case bound is the better choice, matching Section 2.1 (F is small here)

### 4.4 Capacity values: where the theory shows up

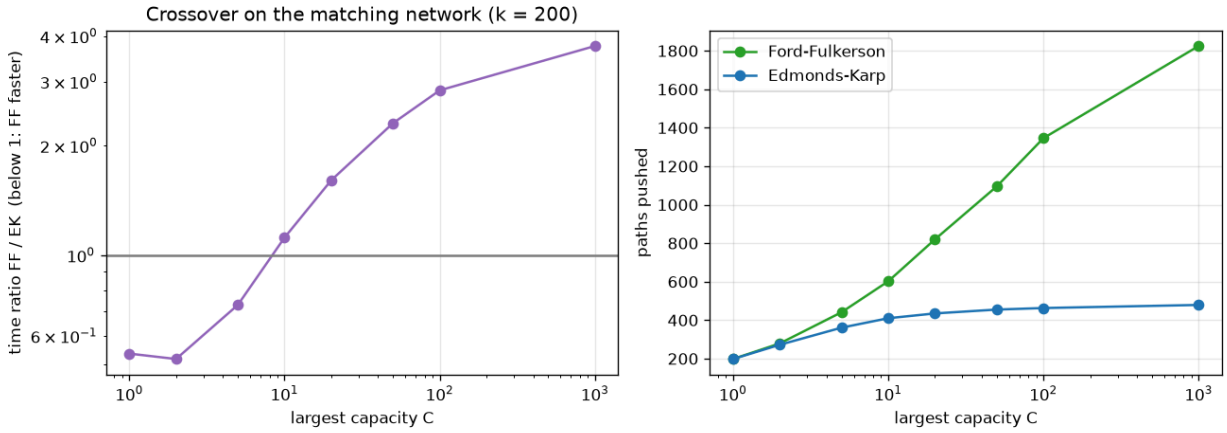
| C (n=300) | FF ms | EK ms | FF/EK | FF paths | EK paths | FF avg path | EK avg path |
|---|---|---|---|---|---|---|---|
| 1 | 1.68 | 0.93 | 1.8 | 3 | 3 | 231.0 | 5.0 |
| 10 | 6.5 | 2.2 | 3.0 | 16 | 9 | 225.1 | 5.7 |
| 100 | 37.5 | 2.8 | 13.3 | 162 | 12 | 221.1 | 6.0 |
| 1000 | 284.3 | 3.0 | 95.1 | 1121 | 9 | 242.3 | 5.6 |
| 10000 | 1199.8 | 2.7 | 452.0 | 4442 | 9 | 244.5 | 5.9 |



- **What it shows:** The graph shape is the same in every row; only the capacity values change. EK pushes between 3 and 12 paths whatever C is. FF's path count grows steeply with C: 162 paths at C = 100, 1,121 at C = 1,000 (about 7 times more) and 4,442 at C = 10,000 (about 4 times more again). At C = 10,000 FF is 452 times slower
- **Why:** This is the contrast between the two bounds. FF's O(FE) grows with the flow value, which grows with the capacities, while EK's O(VE) augmentations depend only on the shape of the graph. Capacities of $10^9$ can force huge numbers of augmentations if paths are chosen badly; my data show the growth already on ordinary random graphs with a plain depth-first search and no adversarial design. At C = 1 both algorithms behave alike (3 paths each)

### 4.5 The crossover on the matching network

| C (k=200) | FF ms | EK ms | FF/EK | FF paths | EK paths | FF scans | EK scans |
|---|---|---|---|---|---|---|---|
| 1 | 15.1 | 28.2 | 0.54 | 198 | 198 | 43,117 | 178,967 |
| 2 | 18.6 | 35.8 | 0.52 | 279 | 272 | 66,125 | 265,888 |
| 5 | 38.2 | 52.3 | 0.73 | 441 | 361 | 137,418 | 400,279 |
| 10 | 67.9 | 60.8 | 1.12 | 602 | 410 | 241,662 | 481,489 |
| 20 | 112.7 | 70.2 | 1.61 | 820 | 435 | 376,888 | 530,649 |
| 50 | 171.1 | 74.3 | 2.30 | 1095 | 455 | 617,977 | 564,766 |
| 100 | 229.4 | 80.9 | 2.84 | 1346 | 463 | 790,628 | 577,407 |
| 1000 | 299.5 | 79.8 | 3.75 | 1825 | 479 | 1,055,202 | 577,535 |




- **What it shows:** This is the experiment that locates the crossover the theme asks for. I take the matching network of Section 4.3 and give every edge a random capacity between

1 and C. At C = 1 FF is almost twice as fast (ratio 0.54). As C grows, FF has to push more and more paths (198 at C = 1, 1,825 at C = 1,000), while EK's count levels off near 480. The ratio crosses 1 near C = 10: FF is faster at C = 5 (0.73), the two are about even at C = 10 (1.12), and EK is clearly faster from C = 20 on. Interpolating between C = 5 and C = 10 puts the crossing near C = 8. Because the run-to-run noise is about 20% (Section 3.7), I state the crossover as "between C = 5 and C = 20, around C = 10"

**4.6 Does a simple cost model explain the timings?**

To check an explanation instead of only proposing it, I tested two models of running time on data the model never saw. I fitted each model on seeds 0 and 1 (136 runs) and measured its error on seeds 2, 3 and 4 (204 runs)

| Model | Fitted numbers | Median error on unseen runs |
|---|---|---|
| A: time = a × arcs scanned | a = 1.51e-07 s per scan | 41% |
| B: time = a × arcs scanned + b × path edges pushed | a = 1.26e-07 s per scan, b = 1.27e-06 s per path edge | 24% |

For the crossover family on the unseen seeds, the measured ratio first exceeds 1 at C = 10. Model A predicts the first crossing at C = 50, model B at C = 20. Adding the length of the pushed paths cuts the typical error from 41% to 24% and moves the predicted crossover much closer to the measured one. The fitted cost of one path edge is about ten times the cost of one scan. The result is generated by experiments/cost_model.py and saved in results/cost_model.md

**4.7 How the predictions fared**

All six predictions held (results/predictions_vs_measured.md). P4 held only just: EK's path count grew exactly 3.0 times, right on the threshold of 3. Because the predictions were informed guesses (Section 2.5), this is a consistency check rather than a test. The informative findings are in Section 5

**4.8 Two checks: how tight the bounds are, and how noisy the timings are**

These two checks were added after the main results and are generated by experiments/bounds.py and experiments/outliers.py (results/bounds.md and results/outliers.md)

- **How tight are the bounds?:** Both bounds hold on all 170 instances, with no violations (FF paths never exceed F, EK paths never exceed VE/2). In practice the two algorithms behave very differently relative to their own bounds. EK uses only a tiny fraction of its bound: on the largest sparse graph it pushes 12 paths against a bound of about 5 million (a fraction of 2.3e-06), and in no family does it use more than about 0.9% of VE/2. FF, in contrast, often reaches its bound. The number of paths equals the flow value F (a ratio of 1.0) on unit-capacity matching (783 paths for F = 783), on the sparse random graphs with 800 or more vertices (182 of 182 and 150 of 150), and on the capacity sweep for C = 1 and C = 10. So in those cases FF pushes about one unit per path. As the capacities grow the ratio falls (0.97 at C = 100, 0.86 at C = 1,000, 0.34 at C = 10,000), but FF still pushes

only about 3 units per path on average at C = 10,000 (13,278 units in 4,442 paths), while EK pushes about 1,500 units per path (13,278 units in 9 paths). In the matching crossover the ratio falls from 1.0 at C = 1 to 0.02 at C = 1,000, and FF still needs 1,825 paths against 479 for EK

- This matches the theory-versus-practice question of the project: FF's capacity-dependent bound is close to what it really does on these inputs, while EK's bound is loose by several orders of magnitude. A likely reason, which I did not test, is that a long depth-first path almost always contains an edge with very little spare capacity, so each push is small. That is also why FF's path count follows F instead of the shape of the graph
- **How noisy are the timings?:** Of the 1,700 timed runs, 1.3% took more than 1.5 times the median of the same instance and algorithm, and 0.1% took more than 2 times. For a typical instance the mean and the median differ by at most 2% (mean divided by median is between 1.00 and 1.02 in every family), so using medians instead of means changes little. Single slow runs do exist: the worst is one Edmonds–Karp run on a dense graph with 100 vertices (seed 0) that took 5.2 times its median. Such runs stretch the min-to-max bars in Figure 1 but do not move the medians, which is why the report states ratios from medians and treats a ratio close to 1 with caution

## 5. What surprised me

Each item gives what I expected, what I measured, the explanation, how I checked it, and what is still open

**Surprise 1:** the crossover arrives long before the arc counts say it should

- **Expected:** My cost idea (Section 2.4) said running time follows the number of arcs scanned. FF scans fewer arcs than EK until about C = 20 to 50 on the matching network, so I expected FF to stay faster until then
- **Measured:** The time ratio crosses 1 between C = 5 and C = 10 (0.73 at C = 5, 1.12 at C = 10). At C = 10 FF still scans only half as many arcs as EK (241,662 against 481,489), yet it is already slower (67.9 ms against 60.8 ms). The scan counts only cross between C = 20 and C = 50 (376,888 against 530,649 at C = 20, 617,977 against 564,766 at C = 50). On all 170 instances, the scan counts and the times still agree in direction in 91% of cases (P6), so the disagreement is concentrated near the crossover
- **Explanation:** Arcs scanned is only half of the work. Every pushed path also costs time in proportion to its length (finding the bottleneck and updating two residual values per edge). At C = 10 FF pushes 602 paths of 19.5 edges on average, about 11,700 path edges, while EK pushes 410 paths of 3.6 edges, about 1,500 path edges. FF's cheaper searches are cancelled out by roughly eight times as much path-pushing
- **How I checked it:** I fitted two models on seeds 0 and 1 and tested them on seeds 2 to 4 (Section 4.6). Counting only scans gives a median error of 41% and predicts the crossover at C = 50. Adding the path edges lowers the error to 24% and predicts the

crossover at C = 20. The fitted cost of a path edge is about ten times that of a scan (1.27e-06 s against 1.26e-07 s)

- **What is still open:** The better model still predicts C = 20, not the measured C = 10, and it still underestimates FF's slowness at C = 10 (predicted ratio 0.75, measured 1.08 on the unseen seeds). I believe the rest is per-search overhead, because every search allocates arrays of length V and FF performs more searches (602 against 410 at C = 10), but I did not test this

**Surprise 2:** depth-first paths are almost as long as the graph is wide

- **Expected:** Prediction P1 only said FF's paths would be at least 2 times longer than EK's
- **Measured:** On the sparse graphs FF's average path has 73 edges at n = 100 and 1,276 edges at n = 1,600, which is 73% and 80% of all vertices. EK's average path is 5.4 to 7.2 edges. The ratio is 177 at n = 1,600, far beyond 2. In the density sweep (n = 60) FF's paths stay at 37 to 43 edges, 62% to 71% of the vertices, for every p from 0.12 to 0.9, while EK's shrink from 5.8 to 2.6
- **Explanation:** A depth-first search takes the first arc it can and only turns back at a dead end. In a random graph almost every vertex can still reach t, so it can wander through most of the graph before it reaches t, and the path it returns is the trail it took. A breadth-first search finds the shortest path, and random graphs have only short shortest paths
- **How I checked it:** The explanation predicts two things I can see in the data: FF's path length should grow in proportion to n, and it should not shrink when the graph gets denser, while EK's should shrink. Both hold (Sections 4.1 and 4.2). What I did not test: whether the order in which arcs are stored matters. Shuffling the arc order is the natural check and is listed under next steps

**Surprise 3:** FF's disadvantage peaks at a medium density and then shrinks
**Expected versus measured:** Prediction P2 only required the ratio to be larger at the densest setting than at the sparsest, and I pictured a steady rise. It is not steady: the time ratio rises from 1.5 (p = 0.03) to 90.3 (p = 0.5) and then falls to 48.7 at p = 0.9. Candidate explanation (not confirmed). Arcs per search (total scans divided by paths + 1, from the table) rise faster for EK than for FF between p = 0.5 and p = 0.9: EK from about 853 to 2,950 (3.5 times), FF from about 1,847 to 3,487 (1.9 times), while the path counts grew by similar factors (FF 990 to 1,613, EK 60 to 107). That fits the idea that a BFS must explore a larger frontier in very dense graphs. But the per-search comparison jumps around at lower densities and I have only 5 seeds on 60 vertices, so I report this as a pattern I can describe but not yet explain

## 6. What I would do next

1. Close the cost-model gap: add a per-search cost term and see whether the predicted crossover moves from C = 20 to the measured C = 10
2. Move the crossover on purpose: write down, before running, where it should move for k = 400 or for twice as many neighbours, then run it and compare
3. Test the arc-order effect on FF's path length by shuffling each vertex's arcs
4. Add a third algorithm that removes the capacity dependence, such as Dinic's algorithm or capacity scaling

## 7. Individual contributions

I am the author and was responsible for both algorithms: ford_fulkerson.py (Ford-Fulkerson with depth-first search) and edmonds_karp.py (Edmonds-Karp with breadth-first search). Beyond the two algorithms, I wrote the paired-arc network data structure, the input generators, the benchmark harness (answer checking before timing, warm-up, alternating run order, garbage collector off), the max-flow = min-cut verification, and the 19 unit tests. I chose the input families, wrote predictions P1 to P6, ran every experiment (170 instances, 1,700 timed runs), fitted and tested the cost models, and wrote all of the report's analysis and text. The AI-generated parts, experiments/bounds.py and experiments/outliers.py, are described in Section 8

## 8. AI use

Tool: Claude

I worked individually and wrote both algorithm implementations, the network data structure, the generators, the benchmark harness, the correctness checks, the unit tests, and the report myself. I used Claude as a reviewer: I gave it my code, report, and experiments and asked it to check them against the assignment requirements. It was most useful for suggesting extra tests and edge cases I had missed. Two files in my submission, experiments/bounds.py and experiments/outliers.py, were generated by Claude. I ran both and checked their output before keeping them. They support Section 4.8: the bounds check found no violations on all 170 instances, and the outlier check showed that only 1.3% of the 1,700 timed runs exceeded 1.5 times their median, which justified my use of medians

**Where AI was unhelpful, and what I did:** First, Claude repeatedly suggested generating more test files, and the result was extension.py, a second benchmark with larger graphs (up to n = 6,400), shifted matching crossovers, and deep layered graphs. I ran and reviewed it, but excluded it because it mostly repeated my crossover, size, and density experiments at other parameters, and it needed a new generator and results format without answering any question the assignment asks. Second, Claude produced a blank blind-prediction template (PREDICTIONS_BLIND.md) to fill in before running that extension. It was not required and served only an experiment I had dropped, so I did not use it. Third, Claude wrote machine_info.py to record hardware details, but "About This Mac" gave me the same information (MacBook Air 2020, 1.1 GHz quad-core Intel Core i5, 8 GB), so I did not add another script

**Reflection:** This assignment taught me not to rely on AI tools to decide what belongs in my work. Claude was good at finding gaps in my testing, but it often added complexity without considering the assignment's scope. For every suggestion, I ran the code, read the output, and tested whether it added real evidence to my comparison before deciding to include it. The bounds and outlier checks did, because they answered real questions about theory versus practice and about how far the timings can be trusted, so I kept them and disclosed them. The extra tests, predictions, and hardware script did not, so I rejected them. Because I wrote the algorithms and experiments myself, I could judge Claude's suggestions against my own understanding instead of trusting them because they looked convincing. I now treat AI output as a proposal to be tested, not a decision